

Introduction

In this report, I have explored different optimisation algorithms applied to three distinct equations, namely, a linear regression task with a quadratic loss surface, a toy neural network with a non-linear tanh activation and a Rosenbrock benchmark function. For each of these functions, loss and gradient are calculated, Hessians are derived wherever possible, and the behaviour of the optimisation functions is studied using the corresponding loss curves and trajectories. These experiments demonstrate some significant trade-offs between convergence speed, stability, gradient noise and computational cost across a range of optimisation techniques.

Question 1 – Linear Regression

The model is $\hat{y} = X\theta$ with squared loss

$$L(\theta) = \frac{1}{2m} \|X\theta - y\|^2, \quad \nabla L(\theta) = \frac{1}{m} X^\top (X\theta - y).$$

Data: $m = 1000$, $X_{ij} \sim \mathcal{N}(0, 1)$, $\theta^* = [3, 4]^\top$, $y = X\theta^* + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$.

Initial point: $\theta_0 = [1, 1]^\top$.

Q1(a)

The full-batch gradient descent is evaluated over all the m samples at every iteration, for 80 iterations, $\alpha = 0.5$ and the parameter is updated as $\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t)$.

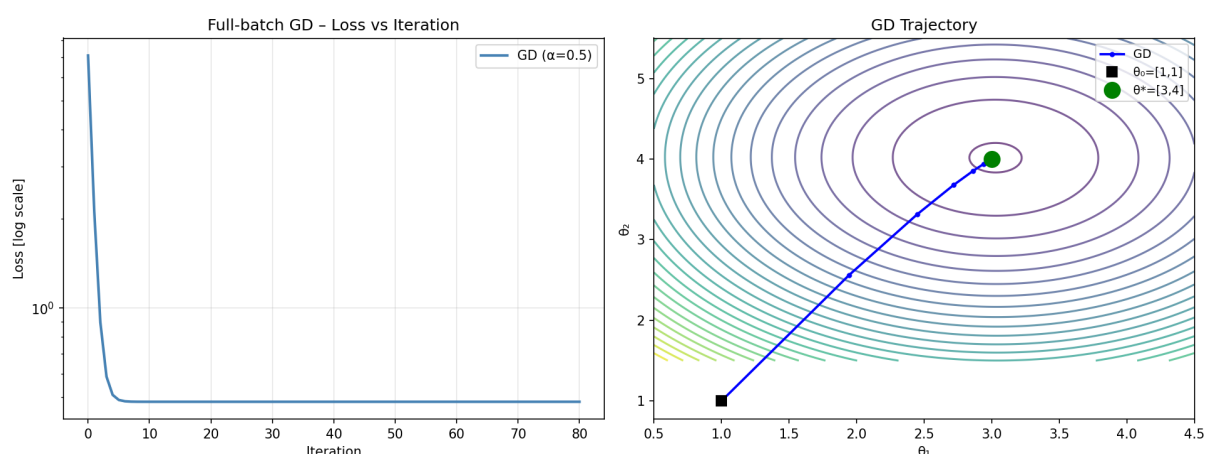


Figure 1: (left) Log-scale loss vs. iteration for full-batch GD; (right) parameter trajectory overlaid on the contour plot of $L(\theta)$.

The plot on the left shows the loss on a log scale, in which the straight downward line confirms the exponential decay at a consistent geometric rate. The plot on the right explores the parameter trajectory by overlaying them on the loss contours, showing how directly the iterations move towards θ^*

Interpretation

- This loss plot (in log-scale) illustrates a monotonous exponential decay over all the 80 iterations, proving that $\alpha = 0.5$ lies below the stability threshold. $1/\lambda_{\max}((1/m)X^T X)$.
- The contour trajectory moves directly toward $\theta^* = [3, 4]$ following the true steepest-descent direction at every step, with no stochastic noise.
- The contour ellipses appear elongated, which shows us the mild ill-conditioning of Hessian and the correlation of X . This becomes the main reason for the slight zig-zags along the major axis in early iterations before it actually stabilises.
- The full-batch GD converges close to θ^* , within 80 iterations, thus demonstrating that the quadratic loss surface of linear regression is suitable for gradient descent when we choose the step size appropriately.

Q1(b)

The Mini-batch SGD is run for 400 updates, $\alpha = 0.5$. Here, the dataset is shuffled at each epoch and partitioned into mini batches of b . The gradient estimated from the mini batch is $g_t = (1/b) X_b^T (X_b \theta_t - y_b)$, $\theta_{t+1} = \theta_t - \alpha g_t$. Two batch sizes are compared namely, $b = 5$ and $b = 20$. The full loss is recorded after every epoch, allowing a fair comparison.

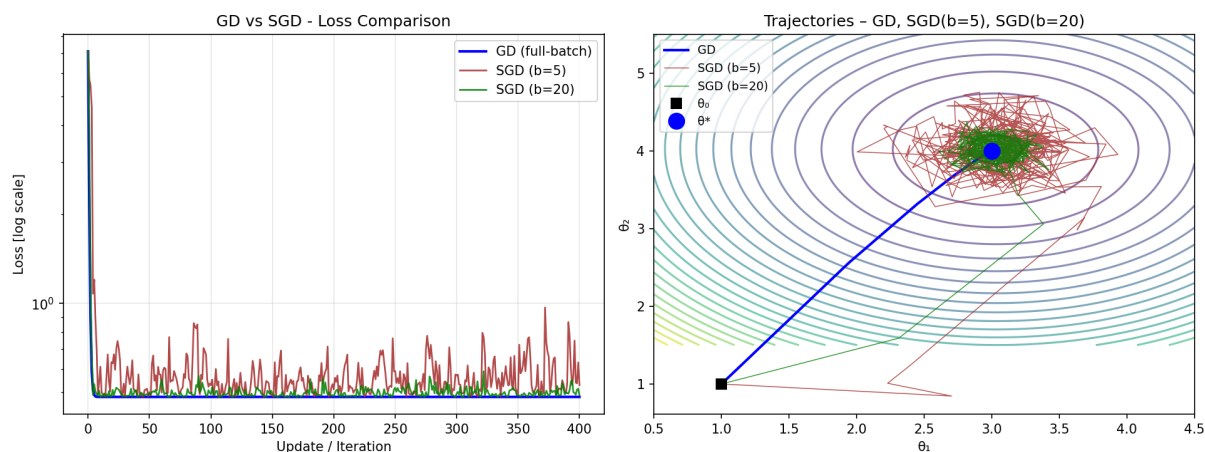


Figure 2: (left) Log-scale loss vs. update for GD, SGD($b = 5$), and SGD($b = 20$); (right) parameter trajectories on the contour plot.

In this graph, the x-axis denotes the parameter updates rather than individual epochs in order to show GD and SGD on an equal footing. Also, the contour plot shows all three trajectories simultaneously, where the effect of gradient noise is visualised by the SGD paths relative to GD.

Interpretation

- Full-batch GD (blue) converges smoothly and has a noiseless reference.

- SGD with $b = 5$ (orange): only 5 samples per gradient estimate produces a high-variance gradient and has an erratic loss curve and a scattered contour plot trajectory.
- SGD with $b = 20$ (green): has a smooth convergence and a lower convergence, and the trajectory stays closer to the direct gradient path.
- Both these SGD variants oscillate around θ^* rather than settling, which proves that it is a characteristic of the constant learning-rate SGD. I understood that a larger batch size reduces the variance and improves trajectory quality at less cost and the optimal batch balances efficiency against cost.

Q1(c)

NAG vs Adagrad comparison is done here for $b = 10$, 400 updates.

NAG ($\beta = 0.9$, $\alpha = 0.5$): uses a momentum variable z and evaluates the gradient at the look-ahead position $\theta_t + \beta z_t$:

$$z_{t+1} = \beta z_t - \alpha \nabla L(\theta_t + \beta z_t), \quad \theta_{t+1} = \theta_t + z_{t+1}.$$

Adagrad ($\alpha = 1.5$): accumulates the sum of squared gradients per dimension:

$$G_{t+1} = G_t + g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\alpha g_t}{\sqrt{G_{t+1}} + \varepsilon}.$$

Here, a higher base rate ($\alpha = 1.5$) is used for Adagrad in order to compensate for its monotonically decaying effective step size.

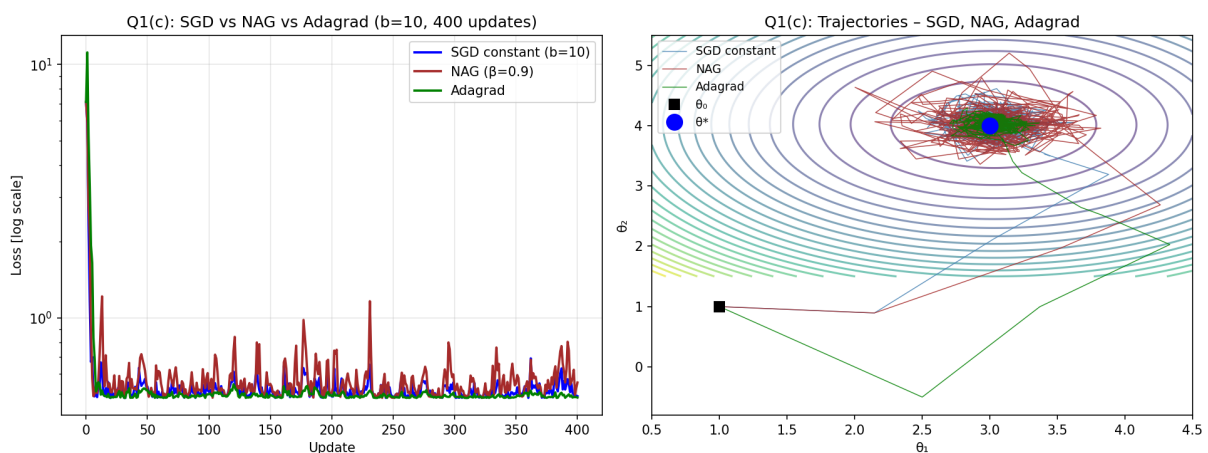


Figure 3: (left) Log-scale loss vs. update for SGD, NAG, and Adagrad ($b = 10$, 400 updates); (right) trajectories on the contour plot.

The relative speed and stability of both the methods are observed from the loss curve. From the contour plot, we can see how close each trajectory follows the valley floor rather than bouncing sideways, thus revealing how both of them handle anisotropic curvature.

Interpretation

- As observed from the graph, NAG converges faster than SGD, because the look-ahead gradient dampens cross-axis oscillations and accelerates the speed along the actual gradient direction.
- Adagrad's per-dimension scaling tends to reduce the steps in steeper direction while keeping the steps in the flatter direction so that the trajectory closely aligns with the valley floor.
- For the given linear regression equation, all three methods converge reliably. The difference in speed and trajectory smoothness highlights the benefit of momentum (NAG) and adaptive scaling (Adagrad) over a plain constant learning rate.
- One trade-off of Adagrad is that its effective learning rate monotonically decreases, which can stall the convergence in very long runs, once the accumulated gradient history becomes too large.

Question 2 – Toy Neural Network

The model is $\hat{y}(u; x) = x_2 \tanh(x_1 u)$ with loss

$$J(x) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (\hat{y}(u_i; x) - y_i)^2.$$

The analytical gradient components are

$$\frac{\partial J}{\partial x_1} = \frac{1}{m} \sum_i r_i x_2 \operatorname{sech}^2(x_1 u_i) u_i, \quad \frac{\partial J}{\partial x_2} = \frac{1}{m} \sum_i r_i \tanh(x_1 u_i),$$

where $r_i = \hat{y}(u_i; x) - y_i$. Data: $m = 1000$, $u_i \sim \text{Uniform}[-2, 2]$, $x^* = [1, 3]^\top$, noise $\sim \mathcal{N}(0, 0.05^2)$. Initial point: $x_0 = [1, 1]^\top$.

Q2(a)

A Full-batch GD is run for 500 iterations, $\alpha = 0.75$. At each iteration, the analytical gradient of J is computed over all 1000 samples. Here, we need more number of iterations compared to the previous question due to the slower convergence on the tanh landscape (non-quadratic)

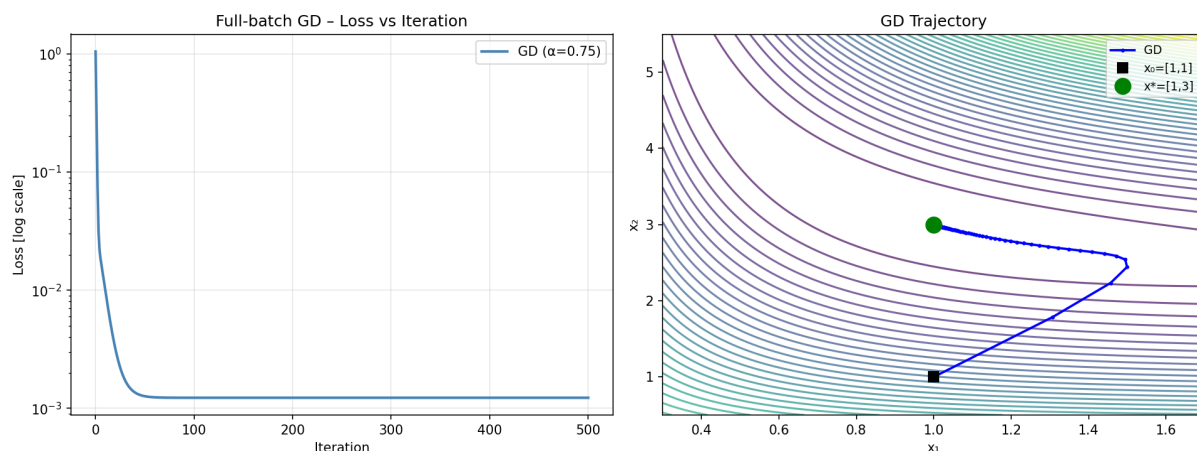


Figure 4: (left) Log-scale loss vs. iteration; (right) trajectory on the contour plot of $J(x)$.

The log-scale loss scale is smooth but it is not perfect, and the varying curvature of the $\tanh$ surface means the decay rate changes along the trajectory. But the contour plot reveals the key geometric challenge, a highly elongated valley along x_2 , which the trajectory must navigate from $x_0 = [1, 1]$ down to $x^* = [1, 3]$.

Interpretation

- It is observed that the loss decreases smoothly, assuring a stable convergence, but the non-constant decay rate shows the non-quadratic nature of $\tanh$ activation.
- The trajectory descends rapidly along the steeper direction first (x_2), then makes slower progress along the flatter direction (x_1) near the optimum, which is seen as the direct consequence of the elongated valley shape.
- GD converges $x^* = [1, 3]$ within 500 iterations, validating the accuracy of the analytical gradient implementation.

Q2(b)

A mini-batch SGD is run at $b = 5$ vs $b = 20$, for 500 updates, $\alpha = 0.75$. The dataset is shuffled at each epoch, and the full dataset loss is recorded after every update. The $\alpha = 0.75$ is remain unchanged for a fair comparison.

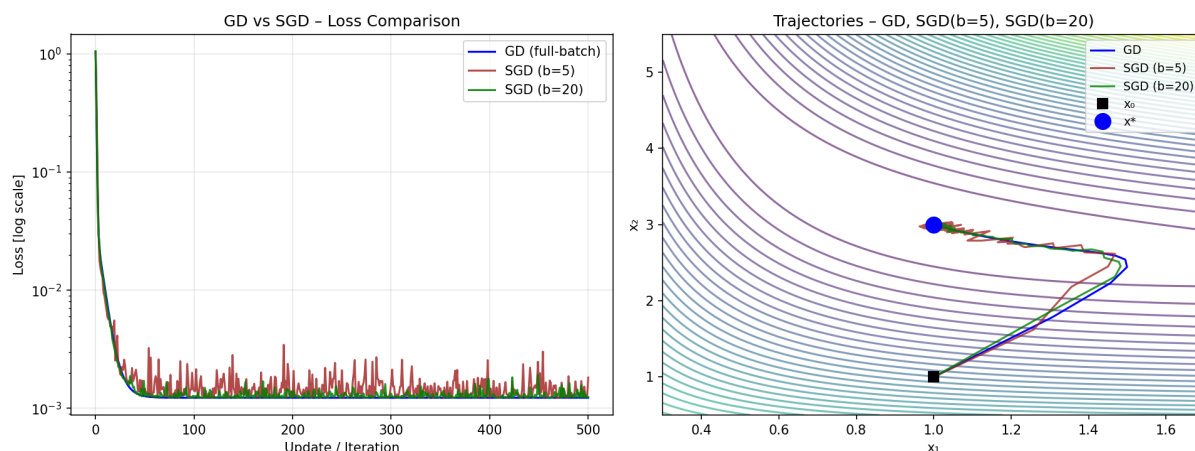


Figure 5: (left) Log-scale loss comparison; (right) parameter trajectories on the contour plot.

The x-axis is kept as parameter updates to put all the methods on equal footing. The contour plot shows a curved, narrow valley, which means SGD trajectories scatter not only around the optimum but also across the valley walls, which is obvious for small batch sizes.

Interpretation

- GD has a smooth and deterministic path along the curved valley floor.
- SGD ($b = 5$): the gradient estimation errors are amplified due to the non-linearity of tanh, resulting in an error-prone loss curve and broader scatters in the contour trajectory.
- SGD ($b = 20$): it has a controlled and a smoother curve, close to the actual descent path, though still noisier than GD.
- The curved valley makes the constant α SGD more problematic because the direction varies along the valley, and so the noisy estimates cause oscillations both along and across the valley.

Q2(c)

NAG vs Adagrad comparison is done for $b = 10$, for 500 updates. NAG ($\beta = 0.9$, $\alpha = 0.5$) and Adagrad ($\alpha = 0.5$) are applied with $b = 10$ mini-batches. The SGD constant α baseline uses $\alpha = 0.75$. A slightly reduced learning rate α is used here for NAG and Adagrad to avoid instability on the curved tanh landscape.

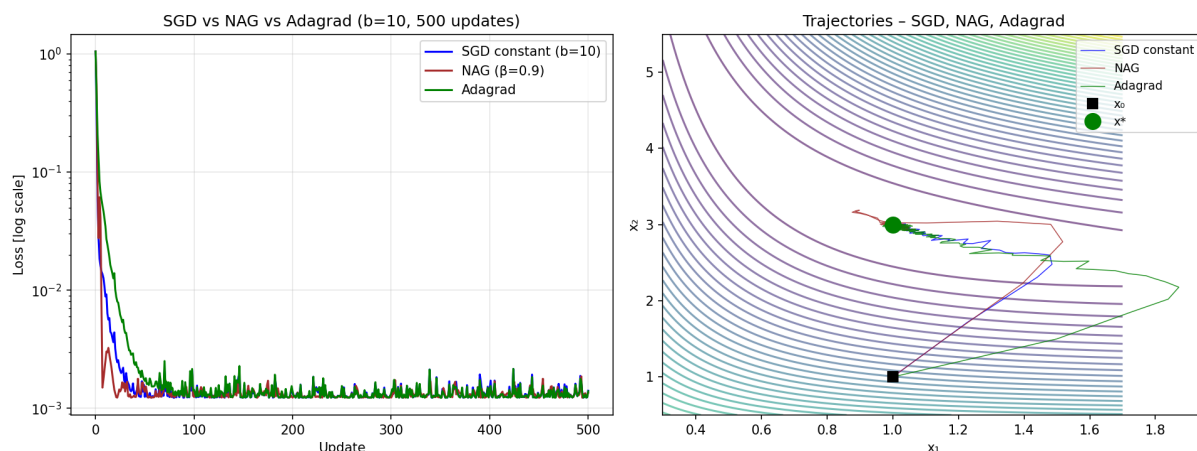


Figure 6: (left) Log-scale loss for SGD, NAG, and Adagrad ($b = 10$, 500 updates); (right) trajectories on the contour plot.

The loss curves demonstrate the speed advantage of NAG and Adagrad over plain SGD. The contour plot shows how each method navigates the elongated curved valley efficiently and how it bounces across the sideways.

Interpretation

- SGD with constant α : a persistent sideways bouncing is observed due to the steep cross-valley curvature of $\tanh$, generating a noisy scattered trajectory despite its overall progress towards the optimum.
- NAG: the momentum accumulates along the consistent descent direction (toward $x_2^* = 3$) and damps cross-valley oscillations via the look-ahead gradient correction, producing a faster and smoother descent than plain SGD. Moreover, occasional overshoot occurs when the valley bends sharply, but it auto-corrects rapidly.
- Adagrad: it is quite effective here because the parameters x_1 x_2 have very different gradient magnitudes in the earlier stage of training. It automatically reduces the effective step for x_2 preventing oscillation while allowing larger relative steps for x_1 , so the trajectory remains closer to the valley floor than the SGD.
- Both NAG and Adagrad outperform constant- α SGD on $\tanh$: NAG exploits gradient history via momentum, while Adagrad exploits per-dimension gradient magnitude information, and all the methods converge to the vicinity of $x^* = [1, 3]$.

Question 3 – Rosenbrock Function

The Rosenbrock function is given as

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2, \quad x^* = [1, 1]^T, \quad f(x^*) = 0.$$

Starting point: $x_0 = [-1, 1]^T$. This function acts as a classical benchmark due to its narrow, curved, banana-shaped valley and extreme ill-conditioning (condition number $\sim 10^3$ – 10^6 near the optimum).

Finite-Difference Formulae

As specified in the lecture notes, the gradient and Hessian are approximated using central differences.

Central-difference gradient ($h = 10^{-5}$):

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + h e_i) - f(x - h e_i)}{2h}.$$

Second-order central-difference Hessian ($h = 10^{-4}$):

$$H_{ij}(x) \approx \frac{f(x + h e_i + h e_j) - f(x + h e_i - h e_j) - f(x - h e_i + h e_j) + f(x - h e_i - h e_j)}{4h^2}.$$

The Newton search direction is $p_t = H(x_t)^{-1} \nabla f(x_t)$ and the update is $x_{t+1} = x_t - \alpha p_t$.

Q3(a)

I've used the standard gradient descent, for 200 iterations, $\alpha = 10^{-3}$, using the exact analytical gradient of the Rosenbrock function. A small step size $\alpha = 10^{-3}$ is chosen to avoid the divergence due to steep cross-valley walls.

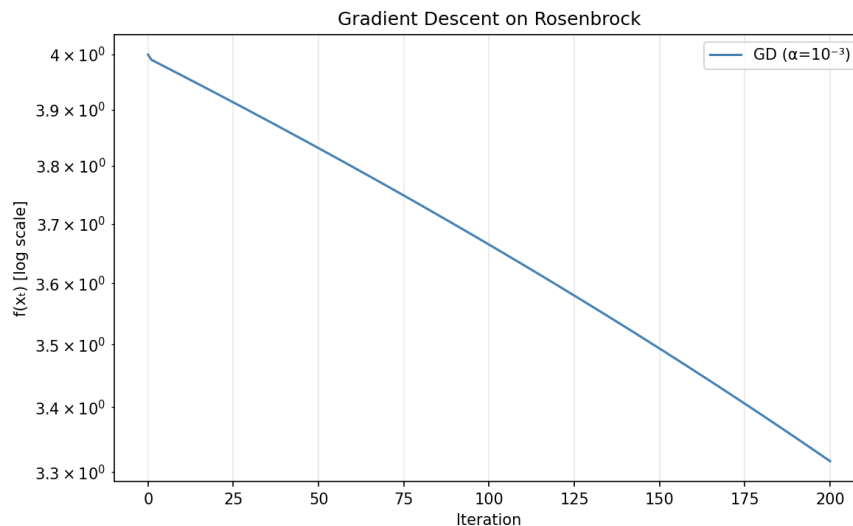


Figure 7: Log-scale loss $f(x_t)$ vs. iteration for gradient descent.

Here, the log-scale curves denotes the overall gradient descent trend, which is quite slow and uneven, and it is the signature of a highly ill-conditioned problem like Rosenbrock.

Interpretation

- It is seen that the curvature across the narrow valley is orders of magnitude higher than the valley floor. Thus choosing a step size safe for the steep direction makes negligible progress along that floor.

- Even after 200 iterations with $\alpha = 10^{-3}$, the iterates are still far from $x^* = [1, 1]$, demonstrating the fundamental limitation of first-order methods on highly ill-conditioned problems.
- Increasing α beyond 10^{-3} causes the iterates to oscillate across the valley walls and diverge, so GD is trapped in slow convergence here.

Q3(b)

Newton's Method with Finite Differences is explored for 200 iterations at $\alpha = 0.7$). At each iteration the FD gradient ($h = 10^{-5}$) and FD Hessian ($h = 10^{-4}$) are computed as specified in the lecture notes. The Newton direction is obtained by solving $(H + \lambda I)p = g$ via `np.linalg.solve`, with regularisation $\lambda = 10^{-6}$ to ensure positive definiteness. The base is updated as $x_{t+1} = x_t - \alpha p_t$

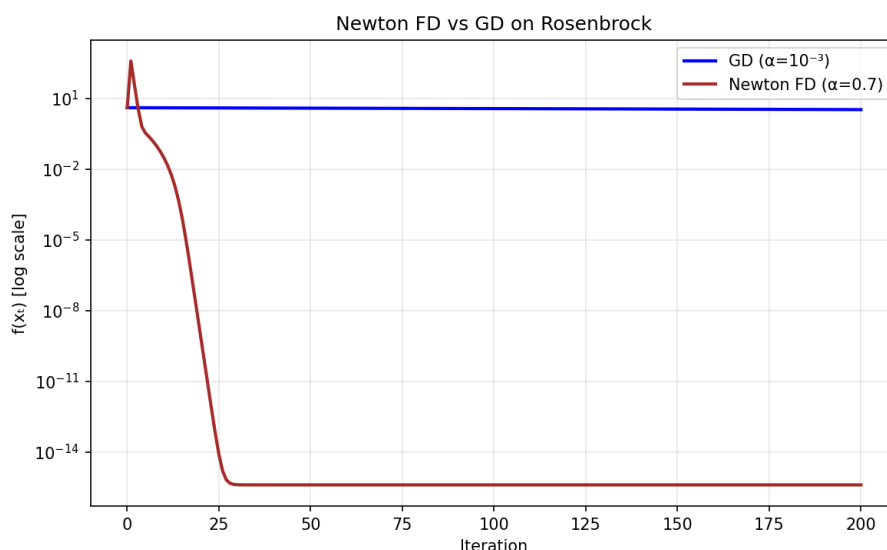


Figure 8: Log-scale loss comparison between GD ($\alpha = 10^{-3}$) and Newton's method with FD approximations ($\alpha = 0.7$).

Plotting the GD and Newton on the same log scale illustrates the stark difference in the convergence gap. It is seen that Newton's curve drops sharply around the initial 30-50 iterations, while GD barely progresses.

Implementation

- The Hessian inverse H^{-1} rescales the gradient to handle the anisotropic curvature by taking the large steps along the flat valley floor and small steps along the steep walls, thus solving the ill-conditioning problem.
- The central difference FD formula is second-order accurate in h , so the numerical approximation errors can be negligible for functions like Rosenbrock.
- Near the optimum, x^* , the loss drop sharply accelerates in the log scale, reflecting

Newton's quadratic convergence rate, i.e., each iteration approximately squares the distance to x^* .

- The fix $\alpha = 0.7$ provides a safety margin in early iterations where the quadratic approximation is less accurate.

Q3(c)

I have explored the Damped Newton with Backtracking Line Search (200 iterations) by following these implementation steps.

1. Compute $f_t, g_t = \nabla_{\text{FD}} f(x_t), H_t = H_{\text{FD}}(x_t)$.
2. Solve $(H_t + \lambda I) p_t = g_t$ for the Newton direction.
3. Set $\alpha = \alpha_0 = 0.7$; repeatedly multiply by $\rho = 0.5$ for up to $K = 20$ steps until $f(x_t - \alpha p_t) < f_t$.
4. If no step is accepted, fall back to $x_{t+1} = x_t - 10^{-4} g_t$.

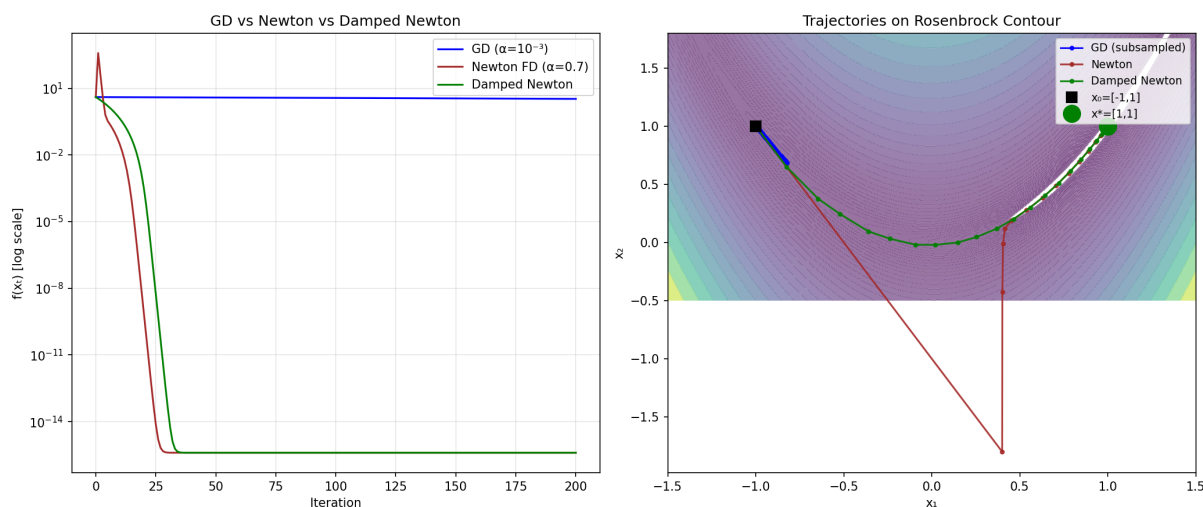


Figure 9: (left) Log-scale loss for GD, Newton, and Damped Newton; (right) contour trajectories for all three methods (GD is subsampled for clarity).

The loss plot compares all the three methods, where the gap between GD and both the Newton methods is obvious. The right contour plot shows that the GD's path bouncing along the valley walls contrasts directly with the clean trajectories of Newton methods, towards $x^* = [1, 1]$.

Interpretation

- It is seen that GD makes nearly negligible progress on the log scale over 200 iterations, whereas the Newton method with fixed α convergence is rapidly (unstable in early iterations). Dampened Newton matches the Newton's speed, giving a monotonous descent at every step.

- The backtracking condition $f(x_t - \alpha p_t) < f_t$ is essential here because the curved banana shape makes the quadratic approximation unreliable far from x^* , and thus a full Newton step can overshoot and increase f .
- When near x^* , both the Newton variants exhibit near-quadratic convergence, which is seen as the sharp final drop in the loss curve; it is basically impossible by any first-order method.
- The fallback step $x - 10^{-4}g$ makes sure that the algorithm does not stall even when the Hessian direction is unreliable (like near the saddle points).
- Overall ranking: Damped Newton $\succ$ Newton (fixed α) $\succ$ GD, in terms of robustness and convergence speed. In well-behaved regions, fixed- α Newton converges equally fast; damped Newton is preferred in practice because it provides a guaranteed decrease at every step.

Appendix: Python Code

Q1 - Linear Regression

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib
4
5 #setup
6 np.random.seed(42)
7 m1 = 1000
8 X1 = np.random.randn(m1, 2)
9 theta_star = np.array([3.0, 4.0])
10 eps1 = np.random.randn(m1)
11 y1 = X1 @ theta_star + eps1
12 theta0 = np.array([1.0, 1.0])
13
14 def compute_loss_q1(theta, X, y):
15     r = X @ theta - y
16     return 0.5 * np.mean(r**2)
17
18 def compute_grad_q1(theta, Xb, yb):
19     r = Xb @ theta - yb
20     return Xb.T @ r / len(yb)
21
22 # contour grid
23 t1_vec = np.linspace(0.5, 4.5, 200)
24 t2_vec = np.linspace(1.5, 5.5, 200)
25 T1, T2 = np.meshgrid(t1_vec, t2_vec)
26 Z1 = np.vectorize(lambda a,b: compute_loss_q1(np.array([a,b]), X1, y1))(
    T1, T2)

```

Q1 (a))

```

1 # Q1 - a
2 def compute_full_gd_q1a(theta0, X, y, alpha, itrs):
3     theta = theta0.copy()
4     losses = [compute_loss_q1(theta, X, y)]
5     trjec = [theta.copy()]
6     for _ in range(itrs):
7         g_q1 = compute_grad_q1(theta, X, y)
8         theta = theta - alpha * g_q1
9         losses.append(compute_loss_q1(theta, X, y))
10        trjec.append(theta.copy())
11    return np.array(losses), np.array(trjec)
12
13 losses_q1a, trjec_q1a = compute_full_gd_q1a(theta0, X1, y1, alpha=0.5,
14        itrs=80)
15
16 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
17 axes[0].semilogy(losses_q1a, color='steelblue', linewidth=2, label='GD (
18     $\alpha$=0.5)')
19 axes[0].set_xlabel('Iteration')
20 axes[0].set_ylabel('Loss [log scale]')

```

```

20 axes[0].set_title('Full-batch GD - Loss vs Iteration')
21 axes[0].legend()
22 axes[0].grid(True, alpha=0.3)
23
24 axes[1].contour(T1, T2, Z1, levels=30, cmap='viridis', alpha=0.7)
25 axes[1].plot(trjec_q1a[:,0], trjec_q1a[:,1], 'b-o', ms=3, lw=1.8, label=
    'GD')
26 axes[1].plot(*theta0, 'ks', ms=8, label='$\theta_0=[1,1]$')
27 axes[1].plot(*theta_star, 'go', ms=12, label='$\theta^*=[3,4]$')
28 axes[1].set_xlabel('$\theta_1$')
29 axes[1].set_ylabel('$\theta_2$')
30 axes[1].set_title('GD Trajectory')
31 axes[1].legend(fontsize=9)
32
33 plt.tight_layout()
34 plt.savefig('q1a.png', dpi=150, bbox_inches='tight')
35 plt.show()

```

Q1 (b))

```

1 #Q1 - b
2 def compute_mbsgd_q1b(theta0, X, y, alpha, n_updts, batch_size, seed=42)
3 :
4     np.random.seed(seed)
5     theta = theta0.copy()
6     n = len(y)
7     losses = [compute_loss_q1(theta, X, y)]
8     trjec = [theta.copy()]
9     count = 0
10    while count < n_updts:
11        idx = np.random.permutation(n)
12        Xs, ys = X[idx], y[idx]
13        for i in range(0, n, batch_size):
14            if count >= n_updts:
15                break
16            Xb, yb = Xs[i:i+batch_size], ys[i:i+batch_size]
17            g = compute_grad_q1(theta, Xb, yb)
18            theta = theta - alpha * g
19            losses.append(compute_loss_q1(theta, X, y))
20            trjec.append(theta.copy())
21            count += 1
22    return np.array(losses), np.array(trjec)
23
24 losses_q1a, trjec_q1a = compute_full_gd_q1a(theta0, X1, y1, alpha=0.5,
25     itr=400)
26
27 losses_q1b_5, trjec_q1b_5 = compute_mbsgd_q1b(theta0, X1, y1, 0.5,
28     400, batch_size=5)
29 losses_q1b_20, trjec_q1b_20 = compute_mbsgd_q1b(theta0, X1, y1, 0.5,
30     400, batch_size=20)
31
32 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
33
34 axes[0].semilogy(losses_q1a, lw=2.0, color='blue', label='GD (full-
    batch)')

```

```

31 axes[0].semilogy(losses_q1b_5, lw=1.3, color='brown', label='SGD (b=5)',
    alpha=0.85)
32 axes[0].semilogy(losses_q1b_20, lw=1.3, color='green', label='SGD (
    b=20)', alpha=0.85)
33 axes[0].set_xlabel('Update / Iteration')
34 axes[0].set_ylabel('Loss [log scale]')
35 axes[0].set_title('GD vs SGD - Loss Comparison')
36 axes[0].legend()
37 axes[0].grid(True, alpha=0.3)
38
39 axes[1].contour(T1, T2, Z1, levels=30, cmap='viridis', alpha=0.6)
40 axes[1].plot(trjec_q1a[:,0], trjec_q1a[:,1], 'b-', lw=2.0, label='GD')
41 axes[1].plot(trjec_q1b_5[:,0], trjec_q1b_5[:,1], '-', lw=0.7, color='
    brown', alpha=0.8, label='SGD (b=5)')
42 axes[1].plot(trjec_q1b_20[:,0], trjec_q1b_20[:,1], '-', lw=0.7, color='
    green', alpha=0.8, label='SGD (b=20)')
43 axes[1].plot(*theta0, 'ks', ms=8, label='$\theta_0$')
44 axes[1].plot(*theta_star, 'bo', ms=12, label='$\theta^*$')
45 axes[1].set_xlabel('$\theta_1$')
46 axes[1].set_ylabel('$\theta_2$')
47 axes[1].set_title('Trajectories -- GD, SGD(b=5), SGD(b=20)')
48 axes[1].legend(fontsize=9)
49
50 plt.tight_layout()
51 plt.savefig('q1b.png', dpi=150, bbox_inches='tight')
52 plt.show()

```

Q1 (c))

```

1 #Q1 - c
2 def compute_nag_sgd_q1c(theta0, X, y, alpha, beta, n_upds, batch_size,
    seed=42):
3     np.random.seed(seed)
4     theta = theta0.copy()
5     z = np.zeros_like(theta)
6     n = len(y)
7     losses = [compute_loss_q1(theta, X, y)]
8     trjec = [theta.copy()]
9     count = 0
10    while count < n_upds:
11        idx = np.random.permutation(n)
12        Xs, ys = X[idx], y[idx]
13        for i in range(0, n, batch_size):
14            if count >= n_upds:
15                break
16            Xb, yb = Xs[i:i+batch_size], ys[i:i+batch_size]
17            theta_look = theta + beta * z
18            g_q1c = compute_grad_q1(theta_look, Xb, yb)
19            z = beta * z - alpha * g_q1c
20            theta = theta + z
21            losses.append(compute_loss_q1(theta, X, y))
22            trjec.append(theta.copy())
23            count += 1
24    return np.array(losses), np.array(trjec)
25

```

```

26
27 def compute_adagrad_sgd_q1c(theta0, X, y, alpha, n_upds, batch_size, eps
    =1e-8, seed=42):
28     np.random.seed(seed)
29     theta = theta0.copy()
30     G = np.zeros_like(theta)
31     n = len(y)
32     losses = [compute_loss_q1(theta, X, y)]
33     traj = [theta.copy()]
34     count = 0
35     while count < n_upds:
36         idx = np.random.permutation(n)
37         Xs, ys = X[idx], y[idx]
38         for i in range(0, n, batch_size):
39             if count >= n_upds:
40                 break
41             Xb, yb = Xs[i:i+batch_size], ys[i:i+batch_size]
42             g = compute_grad_q1(theta, Xb, yb)
43             G += g**2
44             theta = theta - alpha * g / (np.sqrt(G) + eps)
45             losses.append(compute_loss_q1(theta, X, y))
46             traj.append(theta.copy())
47             count += 1
48     return np.array(losses), np.array(traj)
49
50 losses_sgd1_10, traj_sgd1_10 = compute_mbsgd_q1b(theta0, X1, y1, 0.5,
    400, batch_size=10)
51 losses_nag1, traj_nag1 = compute_nag_sgd_q1c(theta0, X1, y1, alpha=0.5,
    beta=0.9, n_upds=400, batch_size=10)
52 losses_ada1, traj_ada1 = compute_adagrad_sgd_q1c(theta0, X1, y1, alpha
    =1.5, n_upds=400, batch_size=10)
53
54
55 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
56
57 axes[0].semilogy(losses_sgd1_10, lw=1.8, color='blue', label='SGD
    constant (b=10)')
58 axes[0].semilogy(losses_nag1, lw=1.8, color='brown', label='NAG ($\beta
    =0.9$)')
59 axes[0].semilogy(losses_ada1, lw=1.8, color='green', label='Adagrad')
60 axes[0].set_xlabel('Update')
61 axes[0].set_ylabel('Loss [log scale]')
62 axes[0].set_title('Q1(c): SGD vs NAG vs Adagrad (b=10, 400 updates)')
63 axes[0].legend()
64 axes[0].grid(True, alpha=0.3)
65
66 axes[1].contour(T1, T2, Z1, levels=30, cmap='viridis', alpha=0.6)
67 axes[1].plot(traj_sgd1_10[:,0], traj_sgd1_10[:,1], '--', lw=0.7, color='
    steelblue', alpha=0.85, label='SGD constant')
68 axes[1].plot(traj_nag1[:,0], traj_nag1[:,1], '--', lw=0.7, color='brown
    ', alpha=0.85, label='NAG')
69 axes[1].plot(traj_ada1[:,0], traj_ada1[:,1], '--', lw=0.7, color='green
    ', alpha=0.85, label='Adagrad')
70 axes[1].plot(*theta0, 'ks', ms=8, label='$\theta_0$')
71 axes[1].plot(*theta_star, 'bo', ms=12, label='$\theta^*$')
72 axes[1].set_xlabel('$\theta_1$')

```

```

73 axes[1].set_ylabel('$\theta_2$')
74 axes[1].set_title('Q1(c): Trajectories -- SGD, NAG, Adagrad')
75 axes[1].legend(fontsize=9)
76
77 plt.tight_layout()
78 plt.savefig('q1c.png', dpi=150, bbox_inches='tight')
79 plt.show()

```

Q2 - Toy Neural Network)

```

1  #setup
2  np.random.seed(42)
3  m2 = 1000
4  u2 = np.random.uniform(-2, 2, m2)
5  x_star_q2 = np.array([1.0, 3.0])
6  eps2 = np.random.randn(m2) * 0.05
7  y2 = x_star_q2[1] * np.tanh(x_star_q2[0] * u2) + eps2
8  x0_q2 = np.array([1.0, 1.0])
9
10
11 def compute_yhat_q2(u, x):
12     return x[1] * np.tanh(x[0] * u)
13
14 def compute_loss_q2(x, u, y):
15     return 0.5 * np.mean((compute_yhat_q2(u, x) - y)**2)
16
17 def compute_grad_q2(x, u, y):
18     r = compute_yhat_q2(u, x) - y
19     th = np.tanh(x[0] * u)
20     sc2 = 1.0 - th**2
21     g1 = np.mean(r * x[1] * sc2 * u)
22     g2 = np.mean(r * th)
23     return np.array([g1, g2])
24
25 #contour grid
26 x1_vec = np.linspace(0.3, 1.7, 200)
27 x2_vec = np.linspace(0.5, 5.5, 200)
28 X1G, X2G = np.meshgrid(x1_vec, x2_vec)
29 Z2 = np.vectorize(lambda a,b: compute_loss_q2(np.array([a,b]), u2, y2))(
    X1G, X2G)

```

Q2 (a)

```

1  #Q2 - a
2  def compute_full_gd_q2a(x0, u, y, alpha, itrs):
3      x = x0.copy()
4      losses = [compute_loss_q2(x, u, y)]
5      trjec = [x.copy()]
6      for _ in range(itrs):
7          g = compute_grad_q2(x, u, y)
8          x = x - alpha * g
9          losses.append(compute_loss_q2(x, u, y))
10         trjec.append(x.copy())

```

```

11     return np.array(losses), np.array(trjec)
12
13 losses_q2a, trjec_q2a = compute_full_gd_q2a(x0_q2, u2, y2, alpha=0.75,
14     itr=500)
15
16 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
17
18 axes[0].semilogy(losses_q2a, color='steelblue', lw=2, label='GD ( $\alpha$ 
19     =0.75)')
20 axes[0].set_xlabel('Iteration')
21 axes[0].set_ylabel('Loss [log scale]')
22 axes[0].set_title('Full-batch GD -- Loss vs Iteration')
23 axes[0].legend()
24 axes[0].grid(True, alpha=0.3)
25 axes[1].contour(X1G, X2G, Z2, levels=40, cmap='viridis', alpha=0.7)
26 axes[1].plot(trjec_q2a[:,0], trjec_q2a[:,1], 'b-o', ms=2, lw=1.8, label=
27     'GD')
28 axes[1].plot(*x0_q2, 'ks', ms=8, label='$x_0=[1,1]$')
29 axes[1].plot(*x_star_q2, 'go', ms=12, label='$x^*=[1,3]$')
30 axes[1].set_xlabel('$x_1$')
31 axes[1].set_ylabel('$x_2$')
32 axes[1].set_title('GD Trajectory')
33 axes[1].legend(fontsize=9)
34
35 plt.tight_layout()
36 plt.savefig('q2a.png', dpi=150, bbox_inches='tight')
37 plt.show()

```

Q2 (b)

```

1 #Q2 - b
2 def compute_minibatch_sgd_q2b(x0, u, y, alpha, n_updts, batch_size, seed
3     =42):
4     np.random.seed(seed)
5     x = x0.copy()
6     n = len(y)
7     losses = [compute_loss_q2(x, u, y)]
8     trjec = [x.copy()]
9     count = 0
10    while count < n_updts:
11        idx = np.random.permutation(n)
12        us, ys = u[idx], y[idx]
13        for i in range(0, n, batch_size):
14            if count >= n_updts:
15                break
16            ub, yb = us[i:i+batch_size], ys[i:i+batch_size]
17            g = compute_grad_q2(x, ub, yb)
18            x = x - alpha * g
19            losses.append(compute_loss_q2(x, u, y))
20            trjec.append(x.copy())
21            count += 1
22    return np.array(losses), np.array(trjec)
23
24 losses_q2a, trjec_q2a = compute_full_gd_q2a(x0_q2, u2, y2, alpha=0.75,
25     itr=500)

```

```

24 losses_q2b_5, trjec_q2b_5 = compute_minibatch_sgd_q2b(x0_q2, u2, y2,
    0.75, 500, batch_size=5)
25 losses_q2b_20, trjec_q2b_20 = compute_minibatch_sgd_q2b(x0_q2, u2, y2,
    0.75, 500, batch_size=20)
26
27
28 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
29
30 axes[0].semilogy(losses_q2a, color='steelblue', label='GD (full-batch)
    ')
31 axes[0].semilogy(losses_q2b_5, color='brown', label='SGD (b=5)',
    alpha=0.85)
32 axes[0].semilogy(losses_q2b_20, lw=1.3, color='green', label='SGD
    (b=20)', alpha=0.85)
33 axes[0].set_xlabel('Update / Iteration')
34 axes[0].set_ylabel('Loss [log scale]')
35 axes[0].set_title('GD vs SGD -- Loss Comparison')
36 axes[0].legend()
37 axes[0].grid(True, alpha=0.3)
38 axes[1].contour(X1G, X2G, Z2, levels=40, cmap='viridis', alpha=0.6)
39 axes[1].plot(trjec_q2a[:,0], trjec_q2a[:,1], 'b-', label='GD')
40 axes[1].plot(trjec_q2b_5[:,0], trjec_q2b_5[:,1], '-', color='brown',
    alpha=0.8, label='SGD (b=5)')
41 axes[1].plot(trjec_q2b_20[:,0], trjec_q2b_20[:,1], '-', color='green',
    alpha=0.8, label='SGD (b=20)')
42 axes[1].plot(*x0_q2, 'ks', ms=8, label='$x_0$')
43 axes[1].plot(*x_star_q2, 'bo', ms=12, label='$x^*$')
44 axes[1].set_xlabel('$x_1$')
45 axes[1].set_ylabel('$x_2$')
46 axes[1].set_title('Trajectories -- GD, SGD(b=5), SGD(b=20)')
47 axes[1].legend(fontsize=9)
48
49 plt.tight_layout()
50 plt.savefig('q2b.png', dpi=150, bbox_inches='tight')
51 plt.show()

```

Q2 (c)

```

1 #Q2 - c
2 def compute_nag_sgd_q2(x0, u, y, alpha, beta, n_updts, batch_size, seed
    =42):
3     np.random.seed(seed)
4     x = x0.copy()
5     z = np.zeros_like(x)
6     n = len(y)
7     losses = [compute_loss_q2(x, u, y)]
8     trjec = [x.copy()]
9     count = 0
10    while count < n_updts:
11        idx = np.random.permutation(n)
12        us, ys = u[idx], y[idx]
13        for i in range(0, n, batch_size):
14            if count >= n_updts:
15                break
16            ub, yb = us[i:i+batch_size], ys[i:i+batch_size]

```

```

17         x_look = x + beta * z
18         g = compute_grad_q2(x_look, ub, yb)
19         z = beta * z - alpha * g
20         x = x + z
21         losses.append(compute_loss_q2(x, u, y))
22         trjec.append(x.copy())
23         count += 1
24     return np.array(losses), np.array(trjec)
25
26
27 def compute_adagrad_sgd_q2(x0, u, y, alpha, n_updts, batch_size, eps=1e
-8, seed=42):
28     np.random.seed(seed)
29     x = x0.copy()
30     G = np.zeros_like(x)
31     n = len(y)
32     losses = [compute_loss_q2(x, u, y)]
33     trjec = [x.copy()]
34     count = 0
35     while count < n_updts:
36         idx = np.random.permutation(n)
37         us, ys = u[idx], y[idx]
38         for i in range(0, n, batch_size):
39             if count >= n_updts:
40                 break
41             ub, yb = us[i:i+batch_size], ys[i:i+batch_size]
42             g = compute_grad_q2(x, ub, yb)
43             G += g**2
44             x = x - alpha * g / (np.sqrt(G) + eps)
45             losses.append(compute_loss_q2(x, u, y))
46             trjec.append(x.copy())
47             count += 1
48     return np.array(losses), np.array(trjec)
49
50 losses_sgd2_10, traj_sgd2_10 = compute_minibatch_sgd_q2b(x0_q2, u2, y2,
0.75, 500, batch_size=10)
51 losses_nag_q2c, traj_nag_q2c = compute_nag_sgd_q2(x0_q2, u2, y2, alpha
=0.5, beta=0.9, n_updts=500, batch_size=10)
52 losses_ada_q2c, traj_ada_q2c = compute_adagrad_sgd_q2(x0_q2, u2, y2,
alpha=0.5, n_updts=500, batch_size=10)
53
54 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
55
56 axes[0].semilogy(losses_sgd2_10, color='blue', label='SGD constant (b
=10)')
57 axes[0].semilogy(losses_nag_q2c, color='brown', label='NAG ($\beta=0.9$)
')
58 axes[0].semilogy(losses_ada_q2c, color='green', label='Adagrad')
59 axes[0].set_xlabel('Update')
60 axes[0].set_ylabel('Loss [log scale]')
61 axes[0].set_title('SGD vs NAG vs Adagrad (b=10, 500 updates)')
62 axes[0].legend()
63 axes[0].grid(True, alpha=0.3)
64 axes[1].contour(X1G, X2G, Z2, levels=40, cmap='viridis', alpha=0.6)
65 axes[1].plot(traj_sgd2_10[:,0], traj_sgd2_10[:,1], '--', lw=0.7, color='
blue', alpha=0.85, label='SGD constant')

```

```

66 axes[1].plot(traj_nag_q2c[:,0], traj_nag_q2c[:,1], '--', lw=0.7, color='
    brown', alpha=0.85, label='NAG')
67 axes[1].plot(traj_ada_q2c[:,0], traj_ada_q2c[:,1], '--', lw=0.7, color='
    green', alpha=0.85, label='Adagrad')
68 axes[1].plot(*x0_q2, 'ks', ms=8, label='$x_0$')
69 axes[1].plot(*x_star_q2, 'go', ms=12, label='$x^*$')
70 axes[1].set_xlabel('$x_1$')
71 axes[1].set_ylabel('$x_2$')
72 axes[1].set_title('Trajectories -- SGD, NAG, Adagrad')
73 axes[1].legend(fontsize=9)
74
75 plt.tight_layout()
76 plt.savefig('q2c.png', dpi=150, bbox_inches='tight')
77 plt.show()

```

Q3 - Rosenbrock Function

```

1 #setup
2 x0_q3 = np.array([-1.0, 1.0])
3 xstar_q3 = np.array([ 1.0, 1.0])
4
5 def get_rosenbrock(x):
6     return (1 - x[0])**2 + 100*(x[1] - x[0]**2)**2
7
8 def compute_rosenbrock_grad(x):
9     g1 = -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
10    g2 = 200*(x[1] - x[0]**2)
11    return np.array([g1, g2])
12
13 xr1 = np.linspace(-1.5, 1.5, 500)
14 xr2 = np.linspace(-0.5, 1.8, 500)
15 XR1, XR2 = np.meshgrid(xr1, xr2)
16 ZR = (1 - XR1)**2 + 100*(XR2 - XR1**2)**2

```

Q3 (a)

```

1 #Q3 - a
2 def compute_gd_rosenbrock(x0, alpha, n_iters):
3     x = x0.copy()
4     losses = [get_rosenbrock(x)]
5     trjec = [x.copy()]
6     for _ in range(n_iters):
7         g = compute_rosenbrock_grad(x)
8         x = x - alpha * g
9         losses.append(get_rosenbrock(x))
10        trjec.append(x.copy())
11    return np.array(losses), np.array(trjec)
12
13 losses_q3a, trjec_q3a = compute_gd_rosenbrock(x0_q3, alpha=1e-3, n_iters
    =200)
14
15 fig, ax = plt.subplots(figsize=(8, 5))
16

```

```

17 ax.semilogy(losses_q3a, color='steelblue', label='GD ( $\alpha=10^{-3}$ )')
18 )
19 ax.set_xlabel('Iteration')
20 ax.set_ylabel('$f(x_t)$ [log scale]')
21 ax.set_title('Gradient Descent on Rosenbrock')
22 ax.legend()
23 ax.grid(True, alpha=0.3)
24
25 plt.tight_layout()
26 plt.savefig('q3a.png', dpi=150, bbox_inches='tight')
27 plt.show()

```

Q3 (b)

```

1 #Q3 - b
2 def compute_fd_grad(f, x, h=1e-5):
3     n = len(x)
4     g = np.zeros(n)
5     for i in range(n):
6         ei = np.zeros(n); ei[i] = 1.0
7         g[i] = (f(x + h*ei) - f(x - h*ei)) / (2*h)
8     return g
9
10 def compute_fd_hess(f, x, h=1e-4):
11     n = len(x)
12     H = np.zeros((n, n))
13     for i in range(n):
14         for j in range(n):
15             ei = np.zeros(n); ei[i] = 1.0
16             ej = np.zeros(n); ej[j] = 1.0
17             fpp = f(x + h*ei + h*ej)
18             fpm = f(x + h*ei - h*ej)
19             fmp = f(x - h*ei + h*ej)
20             fmm = f(x - h*ei - h*ej)
21             H[i, j] = (fpp - fpm - fmp + fmm) / (4*h**2)
22     return H
23
24 def compute_newton_rb(x0, alpha=0.7, itrs=200, lam=1e-6):
25     x = x0.copy()
26     losses = [get_rosenbrock(x)]
27     trjec = [x.copy()]
28     for _ in range(itrs):
29         g = compute_fd_grad(get_rosenbrock, x)
30         H = compute_fd_hess(get_rosenbrock, x)
31         H_reg = H + lam * np.eye(len(x))
32         try:
33             p = np.linalg.solve(H_reg, g)
34         except np.linalg.LinAlgError:
35             p = g
36         x = x - alpha * p
37         losses.append(get_rosenbrock(x))
38         trjec.append(x.copy())
39     return np.array(losses), np.array(trjec)
40

```

```

41 losses_newt_q3b, trjec_newt_q3b = compute_newton_rb(x0_q3, alpha=0.7,
42         itr=200)
43
44 fig, ax = plt.subplots(figsize=(8, 5))
45
46 ax.semilogy(losses_q3a, color='blue', lw=2, label='GD ($\alpha$
47         =10-3)')
48 ax.semilogy(losses_newt_q3b, color='brown', lw=2, label='Newton FD ($\alpha$
49         =0.7)')
50 ax.set_xlabel('Iteration')
51 ax.set_ylabel('$f(x_t)$ [log scale]')
52 ax.set_title('Newton FD vs GD on Rosenbrock')
53 ax.legend()
54 ax.grid(True, alpha=0.3)
55
56 plt.tight_layout()
57 plt.savefig('q3b.png', dpi=150, bbox_inches='tight')
58 plt.show()

```

Q3 (c)

```

1 #Q3 - c
2 def compute_damped_newton_rb(x0, alpha0=0.7, rho=0.5, K=20, itr=200,
3     lam=1e-6):
4     x = x0.copy()
5     losses = [get_rosenbrock(x)]
6     trjec = [x.copy()]
7     for _ in range(itr):
8         ft = get_rosenbrock(x)
9         g = compute_fd_grad(get_rosenbrock, x)
10        H = compute_fd_hess(get_rosenbrock, x)
11        H_reg = H + lam * np.eye(len(x))
12        try:
13            p = np.linalg.solve(H_reg, g)
14        except np.linalg.LinAlgError:
15            p = g
16
17        alpha = alpha0
18        accepted = False
19        for _ in range(K):
20            if get_rosenbrock(x - alpha * p) < ft:
21                accepted = True
22                break
23            alpha *= rho
24
25        x = (x - alpha * p) if accepted else (x - 1e-4 * g)
26        losses.append(get_rosenbrock(x))
27        trjec.append(x.copy())
28    return np.array(losses), np.array(trjec)
29
30 losses_dn_q3c, traj_dn_q3c = compute_damped_newton_rb(x0_q3, alpha0=0.7,
31     rho=0.5, K=20, itr=200)
32
33 fig, axes = plt.subplots(1, 2, figsize=(14, 6))

```

```

33 axes[0].semilogy(losses_q3a, color='blue', label='GD ( $\alpha=10^{-3}$ )')
34 axes[0].semilogy(losses_newt_q3b, color='brown', label='Newton FD ( $\alpha=0.7$ )')
35 axes[0].semilogy(losses_dn_q3c, color='green', label='Damped Newton')
36 axes[0].set_xlabel('Iteration')
37 axes[0].set_ylabel('$f(x_t)$ [log scale]')
38 axes[0].set_title('GD vs Newton vs Damped Newton')
39 axes[0].legend()
40 axes[0].grid(True, alpha=0.3)
41 levels = np.logspace(-0.5, np.log10(ZR.max()), 40)
42 axes[1].contourf(XR1, XR2, ZR, levels=levels, cmap='viridis', alpha=0.55)
43 axes[1].contour(XR1, XR2, ZR, levels=levels, colors='white', linewidths=0.35, alpha=0.4)
44 step = max(1, len(trjec_q3a) // 60)
45 axes[1].plot(trjec_q3a[:,0], trjec_q3a[:,1], 'b-o', ms=3, label='GD (subsampled)', alpha=0.9)
46 axes[1].plot(trjec_newt_q3b[:,0], trjec_newt_q3b[:,1], '-o', color='brown', ms=3, label='Newton', alpha=0.9)
47 axes[1].plot(traj_dn_q3c[:,0], traj_dn_q3c[:,1], '-o', color='green', ms=3, label='Damped Newton', alpha=0.9)
48 axes[1].plot(*x0_q3, 'ks', ms=9, label='$x_0=[-1,1]$')
49 axes[1].plot(*xstar_q3, 'go', ms=14, label='$x^*=[1,1]$')
50 axes[1].set_xlabel('$x_1$')
51 axes[1].set_ylabel('$x_2$')
52 axes[1].set_title('Trajectories on Rosenbrock Contour')
53 axes[1].legend(fontsize=9)
54
55 plt.tight_layout()
56 plt.savefig('q3c.png', dpi=150, bbox_inches='tight')
57 plt.show()

```